- Service Locator (objects and/or functions are centrally registered for runtime access)
- Service Container (manages dependencies in multiple ways)

Both are covered in detail in *PHP Objects, Patterns, and Practice*; I highly recommend obtaining a copy. (Other resources are available, though.)

Structural Patterns

Class design can quickly get out of hand as the complexity of the problem space grows (or perhaps your understanding of the problem space grows). Classes that were once streamlined and focused can begin to accumulate new properties and methods, and elegant interfaces grow into something ugly and difficult to manage. Breaking large classes into smaller, related ones is a good way to tackle interface bloat but introduces new problems around how to manage closely related objects at run time. This is where the structural patterns come in: they provide efficient ways to assemble individual objects into larger structures and ensure the results play nicely with the rest of your code. As before, we'll look at a quick example, and I'll list some other common patterns that you can investigate in your own time.

Let's take the first class of problem mentioned in the preceding paragraph: adding functionality to existing classes in an elegant way. It is not uncommon for classes that model business processes to accumulate extra methods or become split into subclasses as businesses grow and diversify. Imagine we start a pop-up food stall, and it sells one thing: the best pizza you ever tasted. There is only one product: the Pizza, made the same way every time. Business is good, and the software is simple: we represent a unit of pizza (in the app we made for people to place their orders) as a single class: `Pizza`. Over time we expand our offering: blue cheese is very popular, and so too are truffle shavings. Now we sell pizza, with or without blue cheese, and with or without truffle. We could model the various pizzas in our app as the classes `OriginalPizza`, `BlueCheesePizza`, `TrufflePizza`, and `TruffleAndBlueCheesePizza` – if we weren't thinking very hard. Later on, we add more ingredients, creating more permutations of pizza… Very quickly we could end up with dozens of classes, all identical in their interfaces – or worse, differing in their interfaces! – increasing cognitive load on engineers and raising maintenance costs. All for the sake of some extra toppings.

Fortunately, there is a fix for just this scenario, called the *Decorator Pattern*. This pattern defines a single class to represent the top level of the hierarchy (pizza, in our case) and another class to add functionality, called the decorator (or *wrapper*) class.